

Prombot_st125923_A3

October 5, 2025

1 Task 1 & 2: Add metrics function and Ridge regularization to the LogisticRegression class

Repo (Same as A1, A2): <https://github.com/Phyke/AT82.03-ML-Car-Price-Prediction>

```
[49]: import itertools
import time

import matplotlib.pyplot as plt
import mlflow
import numpy as np
import pandas as pd
from dotenv import load_dotenv
from mlflow import MlflowClient
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler
```

```
[ ]: # I deleted the experiment in MLflow server but it just put them into deleted_
    ↪state.
# So I need to search and restore using the ID.

# from mlflow import tracking

# client = tracking.MlflowClient()

# for exp in client.search_experiments(view_type="ALL"):
#     print(exp.experiment_id, exp.name, exp.lifecycle_stage)
```

```
[ ]: # client = MlflowClient()

# client.restore_experiment("655500363000293231")
```

```
[50]: load_dotenv()

mlflow.set_tracking_uri("https://mlflow.ml.brain.cs.ait.ac.th")
```

```
[ ]: # The same data cleaning function as always

def data_cleaning(input_df: pd.DataFrame) -> pd.DataFrame:
    df = input_df.copy()

    # Drop unused columns first to reduce size
    df = df.drop(columns=["torque"])

    # Drop rows that will definitely not be used
    df = df[~df["fuel"].isin(["CNG", "LPG"])]
    df = df[df["owner"] != "Test Drive Car"]

    # Remove Units
    df["mileage"] = df["mileage"].str.split(" ").str[0]
    df["engine"] = df["engine"].str.split(" ").str[0]
    df["max_power"] = df["max_power"].str.split(" ").str[0]

    # Numeric Type conversion
    df["mileage"] = df["mileage"].astype(float)
    df["engine"] = df["engine"].astype(float)
    df["max_power"] = df["max_power"].astype(float)

    # Fix Brand column
    df = df.rename(columns={"name": "brand"})
    df["brand"] = df["brand"].str.split(" ").str[0]

    return df
```

```
[ ]: # Slightly modified preprocessor to not include polynomial features, Everything
↳ else is the same

def get_preprocessor() -> ColumnTransformer:
    # Define transformers for each feature group
    brand_le = (
        "brand_le",
        Pipeline(
            [
                # Pre-imputer for handling missing values before encoding
                ("pre_imputer", SimpleImputer(strategy="most_frequent")),
                ("encoder", OrdinalEncoder(handle_unknown="use_encoded_value",
↳ unknown_value=np.nan)),
```

```

        ("post_imputer", SimpleImputer(strategy="most_frequent")),
        # Post-imputer for handling any NaN introduced by unknown
↳categories during encoding
        # So the pipeline is robust to unseen categories in production
↳data
        # This won't happen if frontend limits the choices to existing
↳categories only
        # But it's better to be safe than sorry
    ],
),
    ["brand"],
)

fuel_le = (
    "fuel_le",
    Pipeline(
        [
            ("pre_imputer", SimpleImputer(strategy="most_frequent")),
            ("encoder", OrdinalEncoder(handle_unknown="use_encoded_value",
↳unknown_value=np.nan)),
            ("post_imputer", SimpleImputer(strategy="most_frequent")),
        ],
    ),
    ["fuel"],
)

transmission_le = (
    "transmission_le",
    Pipeline(
        [
            ("pre_imputer", SimpleImputer(strategy="most_frequent")),
            ("encoder", OrdinalEncoder(handle_unknown="use_encoded_value",
↳unknown_value=np.nan)),
            ("post_imputer", SimpleImputer(strategy="most_frequent")),
        ],
    ),
    ["transmission"],
)

owner_le = (
    "owner_le",
    Pipeline(
        [
            ("pre_imputer", SimpleImputer(strategy="most_frequent")),
            (
                "encoder",
                OrdinalEncoder(

```

```

        categories=[
            [
                "First Owner",
                "Second Owner",
                "Third Owner",
                "Fourth & Above Owner",
            ],
        ],
        handle_unknown="use_encoded_value",
        unknown_value=np.nan,
    ),
    ("post_imputer", SimpleImputer(strategy="most_frequent")),
],
),
["owner"],
)

seller_type_ohe = (
    "seller_type_ohe",
    Pipeline(
        [
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("encoder", OneHotEncoder(sparse_output=False, drop="first",
↪handle_unknown="ignore")),
        ],
    ),
    ["seller_type"],
)

pipeline_steps = [
    (
        "imputer",
        ColumnTransformer(
            transformers=[
                ("year_imputer", SimpleImputer(strategy="median"),
↪["year"]),
                ("km_imputer", SimpleImputer(strategy="median"),
↪["km_driven"]),
                ("mileage_imputer", SimpleImputer(strategy="mean"),
↪["mileage"]),
                ("engine_imputer", SimpleImputer(strategy="median"),
↪["engine"]),
                ("max_power_imputer", SimpleImputer(strategy="median"),
↪["max_power"]),

```

```

        ("seats_imputer", SimpleImputer(strategy="most_frequent"),
        ↪["seats"])),
        ],
        remainder="passthrough",
    ),
    ("scaler", StandardScaler()),
]

num_scaler = (
    "num_scaler",
    Pipeline(pipeline_steps,
    ["year", "km_driven", "mileage", "engine", "max_power", "seats"],
)

# Combine all transformers into the ColumnTransformer
return ColumnTransformer(
    transformers=[
        brand_le,
        fuel_le,
        transmission_le,
        owner_le,
        seller_type_ohe,
        num_scaler,
    ],
    remainder="passthrough",
    verbose_feature_names_out=True,
)

```

```

[ ]: # Load, Clean, Split, Preprocess Data
df = pd.read_csv("Cars.csv")
df_train, df_test = train_test_split(df, random_state=42, test_size=0.2)
df_train = data_cleaning(df_train)
df_test = data_cleaning(df_test)
X_train = df_train.drop(columns=["selling_price"])
X_test = df_test.drop(columns=["selling_price"])
y_train = df_train["selling_price"]
y_test = df_test["selling_price"]
# The preprocessor here is for first fitting and experimenting in jupyter
↪notebook only
preprocessor = get_preprocessor()
X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

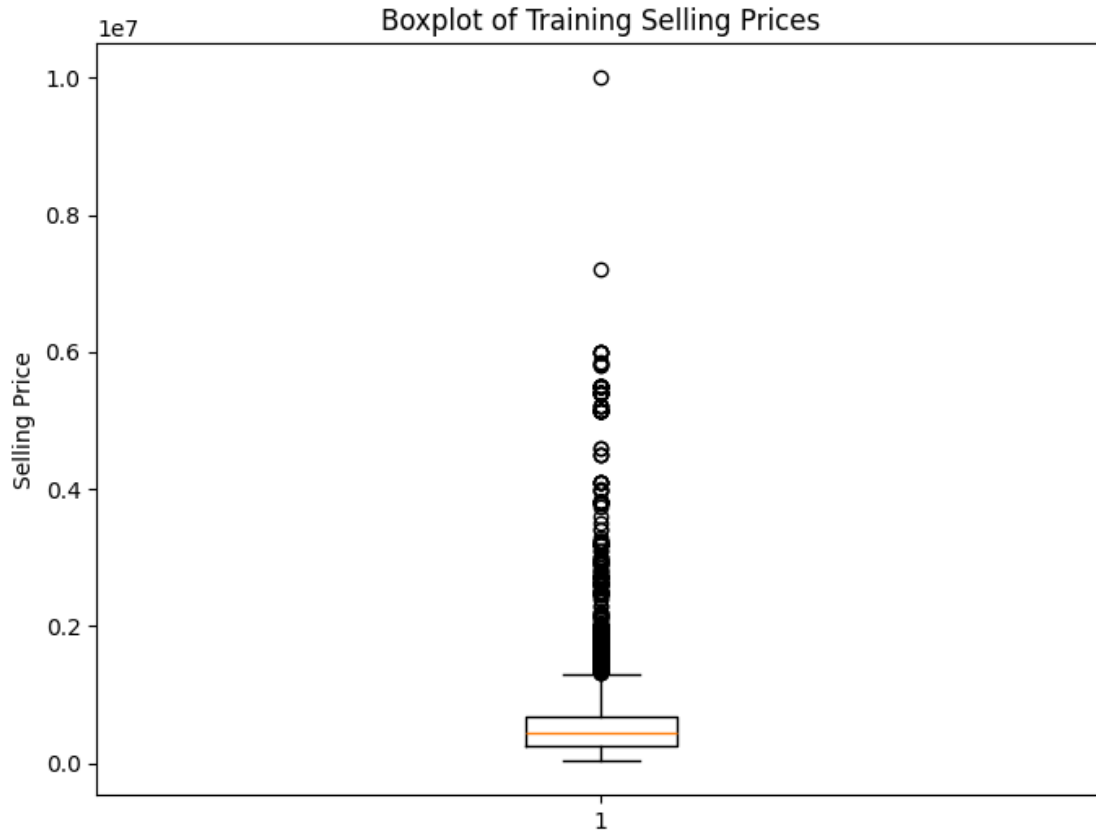
```

```

[ ]: # To separate selling price into classes for classification,
# I need to see the distribution of selling prices first
plt.figure(figsize=(8, 6))

```

```
plt.boxplot(y_train)
plt.title("Boxplot of Training Selling Prices")
plt.ylabel("Selling Price")
plt.show()
```



```
[ ]: # I use the quartiles to define the classes
# Below Q1 is class 0
# Q1 to Q2 is class 1
# Q2 to Q3 is class 2
# Above Q3 is class 3
q1 = y_train.quantile(0.25)
q2 = y_train.quantile(0.5)
q3 = y_train.quantile(0.75)
print(f"Q1: {q1}, Q2: {q2}, Q3: {q3}")
```

Q1: 260000.0, Q2: 450000.0, Q3: 685000.0

```
[ ]: # Categorize the selling prices into 4 classes
y_train_class = pd.cut(
    y_train,
```

```

        bins=[-1, q1, q2, q3, np.inf],
        labels=[0, 1, 2, 3],
    )
    y_test_class = pd.cut(
        y_test,
        bins=[-1, q1, q2, q3, np.inf],
        labels=[0, 1, 2, 3],
    )

```

```

[ ]: # Check the distribution of classes
     y_train_class.value_counts()

```

```

[ ]: selling_price
0    1638
1    1616
3    1603
2    1564
Name: count, dtype: int64

```

```

[59]: y_train_class

```

```

[59]: 6518    2
      6144    1
      6381    1
      438    2
      5939    1
      ..
      5226    2
      5390    2
      860    2
      7603    3
      7270    0
Name: selling_price, Length: 6421, dtype: category
Categories (4, int64): [0 < 1 < 2 < 3]

```

```

[ ]: # From my previous assignment, I was too adventurous and added type annotations
     ↳ to everything
     # Here I remove them to make the code compatible on any system, especially for
     ↳ the Mlflow server

class LogisticRegression:
    def __init__(self, train_method, alpha, max_iter):
        self.alpha = alpha
        self.max_iter = max_iter
        self.train_method = train_method

```

```

def fit(self, X, Y):
    # Initialize weights randomly
    X_intercept = self.add_intercept(X)

    # Convert Y to one-hot encoding
    Y = self.to_one_hot(Y)

    # Initialize weights (theta but matrix form)
    self.n = X_intercept.shape[1]
    self.k = Y.shape[1]
    self.W = np.random.rand(self.n, self.k)
    self.losses = []

    # Here I separate the training methods into different functions
    # So it doesn't clutter the fit function
    if self.train_method == "batch":
        self.batch_gradient(X_intercept, Y)
    elif self.train_method == "mini-batch":
        self.minibatch_gradient(X_intercept, Y)
    elif self.train_method == "stochastic":
        self.stochastic_gradient(X_intercept, Y)
    else:
        msg = (
            'Method must be one of the followings: "batch", "mini-batch" or ↪
            ↪"stochastic". '
            "The provided method was + self.train_method"
        )
        raise ValueError(msg)

def batch_gradient(self, X, Y):
    start_time = time.time()
    for i in range(self.max_iter):
        loss, grad = self.gradient(X, Y)
        self.losses.append(loss)
        self.W = self.W - self.alpha * grad
        if i % 500 == 0:
            print(f"Loss at iteration {i}", loss)
    print(f"time taken: {time.time() - start_time}")

def minibatch_gradient(self, X, Y):
    start_time = time.time()
    batch_size = int(0.1 * X.shape[0])
    for i in range(self.max_iter):
        # For this random sampling with replacement,
        # I reimplemented it because the old one just random 1 index
        # and select a a patch of batch_size Like this [idx: idx + ↪
        ↪batch_size]

```



```

        # So it might be smaller than batch_size if idx is near the end of
        ↪ the array
        # but this way it's always exactly batch_size
        batch_indices = np.random.choice(X.shape[0], size=batch_size,
        ↪ replace=True)
        batch_X = X[batch_indices]
        batch_Y = Y[batch_indices]

        loss, grad = self.gradient(batch_X, batch_Y)
        self.losses.append(loss)
        self.W = self.W - self.alpha * grad
        if i % 500 == 0:
            print(f"Loss at iteration {i}", loss)
        print(f"time taken: {time.time() - start_time}")

    def stochastic_gradient(self, X, Y):
        start_time = time.time()
        for i in range(self.max_iter):
            # For this random sampling with replacement,
            # I reimplemented it because the old one has weird logic which hard
            ↪ to understand
            # So I just made it simple by randomly select 1 index
            idx = np.random.randint(X.shape[0])
            X_train = X[idx, :].reshape(1, -1)
            Y_train = Y[idx]
            loss, grad = self.gradient(X_train, Y_train)
            self.losses.append(loss)
            self.W = self.W - self.alpha * grad
            if i % 500 == 0:
                print(f"Loss at iteration {i}", loss)
            print(f"time taken: {time.time() - start_time}")

    def gradient(self, X, Y):
        Y_hat = self.h_theta(X, self.W)
        loss = self.cross_entropy(Y, Y_hat)
        grad = X.T @ (Y_hat - Y) / Y.shape[0]
        return loss, grad

    def cross_entropy(self, Y, Y_hat):
        m = Y.shape[0]
        return -np.sum(Y * np.log(Y_hat)) / m

    def softmax(self, theta_t_x):
        return np.exp(theta_t_x) / np.sum(np.exp(theta_t_x), axis=1,
        ↪ keepdims=True)

    # Add intercept term to feature matrix

```

```

def add_intercept(self, X):
    intercept = np.ones((X.shape[0], 1))
    return np.hstack((intercept, X))

def to_one_hot(self, Y):
    # Just use simple mapping since there are only 4 classes and avoid
    ↪ sklearn dependency
    oh_mapping = {
        "0": [1, 0, 0, 0],
        "1": [0, 1, 0, 0],
        "2": [0, 0, 1, 0],
        "3": [0, 0, 0, 1],
    }
    return np.array([oh_mapping[str(label)] for label in Y])

def h_theta(self, X, W):
    """
    Input:
        X shape: (m, n)
        w shape: (n, k)
    Returns:
        yhat shape: (m, k)
    """
    return self.softmax(X @ W)

def predict(self, X_test):
    X_test_intercept = self.add_intercept(X_test)
    probabilities = self.h_theta(X_test_intercept, self.W)
    return np.argmax(probabilities, axis=1)

def predict_proba(self, X_test):
    X_test_intercept = self.add_intercept(X_test)
    return self.h_theta(X_test_intercept, self.W)

def plot(self):
    plt.plot(np.arange(len(self.losses)), self.losses, label="Train Losses")
    plt.title("Losses")
    plt.xlabel("epoch")
    plt.ylabel("losses")
    plt.legend()

def true_positives(self, y_true, y_pred, label):
    return np.sum((y_true == label) & (y_pred == label))

def true_negatives(self, y_true, y_pred, label):
    return np.sum((y_true != label) & (y_pred != label))

```

```

def false_positives(self, y_true, y_pred, label):
    return np.sum((y_true != label) & (y_pred == label))

def false_negatives(self, y_true, y_pred, label):
    return np.sum((y_true == label) & (y_pred != label))

def accuracy(self, y_true, y_pred):
    return np.mean(y_true == y_pred)

def precision(self, y_true, y_pred, label):
    tp = self.true_positives(y_true, y_pred, label)
    fp = self.false_positives(y_true, y_pred, label)
    if (tp + fp) == 0:
        return 0.0
    return tp / (tp + fp)

def recall(self, y_true, y_pred, label):
    tp = self.true_positives(y_true, y_pred, label)
    fn = self.false_negatives(y_true, y_pred, label)
    if (tp + fn) == 0:
        return 0.0
    return tp / (tp + fn)

def f1_score(self, y_true, y_pred, label):
    precision = self.precision(y_true, y_pred, label)
    recall = self.recall(y_true, y_pred, label)
    if (precision + recall) == 0:
        return 0.0
    return 2 * (precision * recall) / (precision + recall)

def macro_precision(self, y_true, y_pred):
    labels = np.unique(y_true)
    precisions = [self.precision(y_true, y_pred, label) for label in labels]
    return np.mean(precisions)

def macro_recall(self, y_true, y_pred):
    labels = np.unique(y_true)
    recalls = [self.recall(y_true, y_pred, label) for label in labels]
    return np.mean(recalls)

def macro_f1_score(self, y_true, y_pred):
    labels = np.unique(y_true)
    f1s = [self.f1_score(y_true, y_pred, label) for label in labels]
    return np.mean(f1s)

def weighted_precision(self, y_true, y_pred):
    labels, counts = np.unique(y_true, return_counts=True)

```

```

        precisions = [self.precision(y_true, y_pred, label) for label in labels]
        weighted_precisions = [p * c for p, c in zip(precisions, counts,
↪strict=False)]
        return np.sum(weighted_precisions) / np.sum(counts)

    def weighted_recall(self, y_true, y_pred):
        labels, counts = np.unique(y_true, return_counts=True)
        recalls = [self.recall(y_true, y_pred, label) for label in labels]
        weighted_recalls = [r * c for r, c in zip(recalls, counts,
↪strict=False)]
        return np.sum(weighted_recalls) / np.sum(counts)

    def weighted_f1_score(self, y_true, y_pred):
        labels, counts = np.unique(y_true, return_counts=True)
        f1s = [self.f1_score(y_true, y_pred, label) for label in labels]
        weighted_f1s = [f1 * c for f1, c in zip(f1s, counts, strict=False)]
        return np.sum(weighted_f1s) / np.sum(counts)

    def classification_report(self, y_true, y_pred):
        labels = np.unique(y_true)
        header = "| Label | Precision | Recall | F1-Score_|
↪|\n|-----|-----|-----|-----|"
        print(header)
        for label in labels:
            precision = self.precision(y_true, y_pred, label)
            recall = self.recall(y_true, y_pred, label)
            f1 = self.f1_score(y_true, y_pred, label)
            print(f"| {label} | {precision:.4f} | {recall:.4f} | {f1:.4f} |")
        print(
            f"| Macro Avg | {self.macro_precision(y_true, y_pred):.4f} | "
            f"{self.macro_recall(y_true, y_pred):.4f} | "
            f"{self.macro_f1_score(y_true, y_pred):.4f} |",
        )
        print(
            f"| Weighted Avg | {self.weighted_precision(y_true, y_pred):.4f} | "
            f"{self.weighted_recall(y_true, y_pred):.4f} | "
            f"{self.weighted_f1_score(y_true, y_pred):.4f} |",
        )
        print(f"\n**Overall Accuracy:** {self.accuracy(y_true, y_pred):.4f}")

class RidgePenalty:
    def __init__(self, l):
        self.l = l

    def __call__(self, theta):
        return self.l * np.sum(np.square(theta[1:]))

```

```

def derivation(self, theta):
    g = np.zeros_like(theta)
    g[1:] = self.l * 2 * (theta[1:])
    return g

class RidgeLogisticRegression(LogisticRegression):
    def __init__(self, train_method, alpha, max_iter, l=0.1):
        super().__init__(train_method, alpha, max_iter)
        self.penalty = RidgePenalty(l)

    def gradient(self, X, Y):
        m = X.shape[0]
        Y_hat = self.h_theta(X, self.W)
        loss = -np.sum(Y * np.log(Y_hat)) / m + self.penalty(self.W) / m
        grad = X.T @ (Y_hat - Y) + self.penalty.derivation(self.W) / m
        return loss, grad

```

```

[61]: k = len(y_train_class.unique())
      m = X_train_processed.shape[1]
      model = LogisticRegression("batch", 0.001, 5000)

```

```

[62]: model.fit(X_train_processed, y_train_class)
      y_pred = model.predict(X_test_processed)
      y_pred

```

```

Loss at iteration 0 4.536297068477687
Loss at iteration 500 1.5409429825202146
Loss at iteration 1000 1.3657428717422775
Loss at iteration 1500 1.2511066571578204
Loss at iteration 2000 1.1740176952339256
Loss at iteration 2500 1.1196072063237255
Loss at iteration 3000 1.0792089719510243
Loss at iteration 3500 1.0478301390435032
Loss at iteration 4000 1.0225255229656818
Loss at iteration 4500 1.0014938374248081
time taken: 2.254272222518921

```

```

[62]: array([0, 1, 1, ..., 1, 1, 0], shape=(1607,))

```

```

[63]: y_pred

```

```

[63]: array([0, 1, 1, ..., 1, 1, 0], shape=(1607,))

```

```

[64]: model.classification_report(y_test_class, y_pred)

```

```

| Label | Precision | Recall | F1-Score |
|-----|-----|-----|-----|

```

0	0.7127	0.8010	0.7543	
1	0.4720	0.4533	0.4625	
2	0.4052	0.3204	0.3579	
3	0.6066	0.6816	0.6419	
Macro Avg	0.5491	0.5641	0.5541	
Weighted Avg	0.5495	0.5644	0.5545	

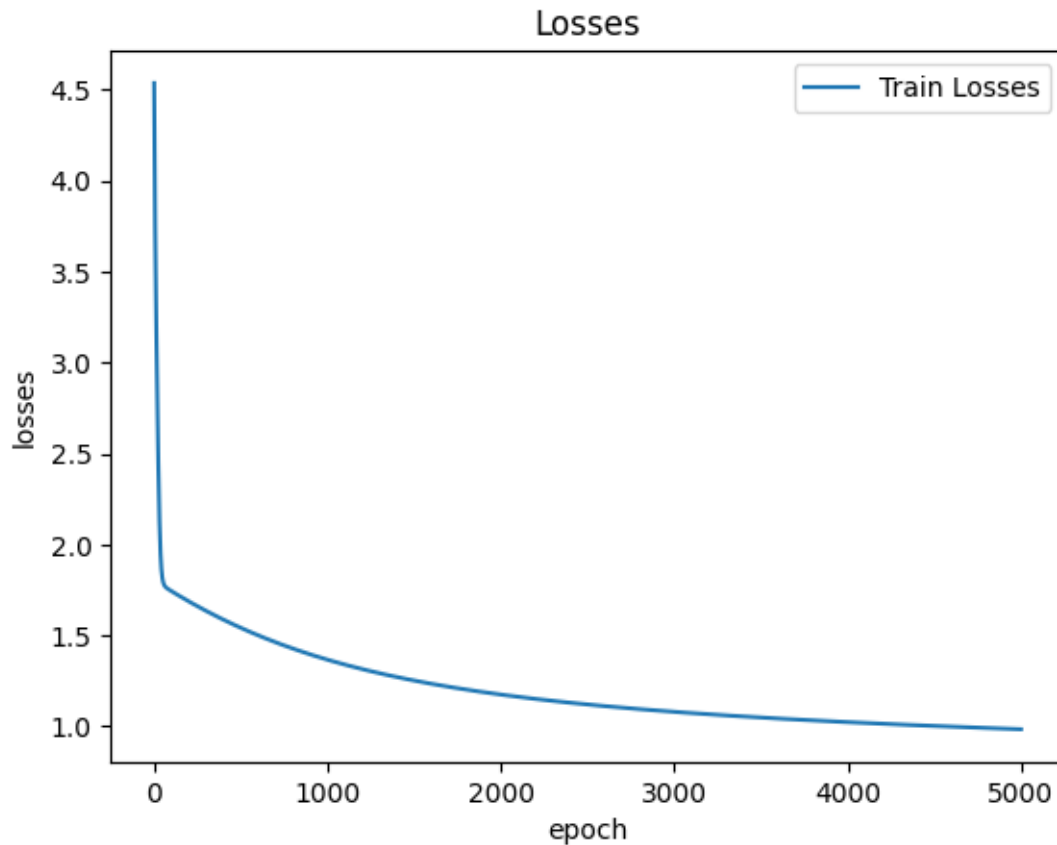
****Overall Accuracy:** 0.5644**

```
[65]: model.plot()
print("=====Classification report=====")
print(classification_report(y_test_class, y_pred))
```

```
=====Classification report=====
              precision    recall  f1-score   support

     0           0.71         0.80         0.75         412
     1           0.47         0.45         0.46         428
     2           0.41         0.32         0.36         387
     3           0.61         0.68         0.64         380

 accuracy                   0.56         1607
 macro avg              0.55         0.56         0.55         1607
 weighted avg           0.55         0.56         0.55         1607
```



What does support in the classification report means? - Support is a number of samples the respective class

2 Task 3: Deployment

```
[67]: model_types = ["Normal", "Ridge"]
train_methods = ["batch", "mini-batch", "stochastic"]
learning_rates = [0.01, 0.001]

all_combinations = list(
    itertools.product(
        model_types,
        train_methods,
        learning_rates,
    ),
)
keys = ["model_type", "train_method", "learning_rate"]
params_combinations = [dict(zip(keys, comb, strict=False)) for comb in
    ↪all_combinations]
```

```
len(params_combinations)
```

[67]: 12

```
[68]: params_combinations[0]
```

```
[68]: {'model_type': 'Normal', 'train_method': 'batch', 'learning_rate': 0.01}
```

```
[69]: def start_experiment(
    X_train_original: pd.DataFrame,
    y_train_original: pd.Series,
    X_test: pd.DataFrame,
    y_test: pd.Series,
    params_combinations: list[dict],
    max_iter: int = 5000,
) -> dict:
    # models = {}
    for i, params in enumerate(params_combinations):
        log_string =
        ↪ f"Reg-{params['model_type']}_TrainMethod-{params['train_method']}_LR-{params['learning_rate']}"
        mlflow.start_run(run_name=log_string)
        mlflow.log_params(params)

        print("===== Experiment: ", i + 1, " / ",
        ↪ len(params_combinations), "=====")
        print(log_string)

        if params["model_type"] == "Normal":
            model = LogisticRegression(
                train_method=params["train_method"],
                alpha=params["learning_rate"],
                max_iter=max_iter,
            )
        elif params["model_type"] == "Ridge":
            model = RidgeLogisticRegression(
                train_method=params["train_method"],
                alpha=params["learning_rate"],
                max_iter=max_iter,
                l=0.1,
            )
        else:
            print("Error: Please select correct model type.")
            break

    pipeline = Pipeline(
        steps=[
            ("preprocess", get_preprocessor()),
```



```

        ("model", model),
    ],
)

# After adding all the steps, fit the pipeline
# The reason to fit the whole pipeline here instead of just the model
# is to ensure that the shape of X after preprocessing will match the
→ input of model
    # so that during prediction, I can just call pipeline.predict(X)
→ directly
    # using the original X_train without any preprocessing
    # which will be useful during inference in production where input data
→ is raw but human-readable
    pipeline.fit(X_train_original, y_train_original)

    # models[log_string] = pipeline

y_pred = pipeline.predict(X_test)

# Get model from pipeline
# (last step so index = -1)
# and it's a tuple of (name, model) so get the model with [1]
fitted_model = pipeline.steps[-1][1]

pipeline.steps[-1][1].classification_report(y_test, y_pred)

mlflow.log_metrics(
    metrics={
        "accuracy": fitted_model.accuracy(y_test, y_pred),
        "class_0_precision": fitted_model.precision(y_test, y_pred, 0),
        "class_0_recall": fitted_model.recall(y_test, y_pred, 0),
        "class_0_f1": fitted_model.f1_score(y_test, y_pred, 0),
        "class_1_precision": fitted_model.precision(y_test, y_pred, 1),
        "class_1_recall": fitted_model.recall(y_test, y_pred, 1),
        "class_1_f1": fitted_model.f1_score(y_test, y_pred, 1),
        "class_2_precision": fitted_model.precision(y_test, y_pred, 2),
        "class_2_recall": fitted_model.recall(y_test, y_pred, 2),
        "class_2_f1": fitted_model.f1_score(y_test, y_pred, 2),
        "class_3_precision": fitted_model.precision(y_test, y_pred, 3),
        "class_3_recall": fitted_model.recall(y_test, y_pred, 3),
        "class_3_f1": fitted_model.f1_score(y_test, y_pred, 3),
        "macro_precision": fitted_model.macro_precision(y_test, y_pred),
        "macro_recall": fitted_model.macro_recall(y_test, y_pred),
        "macro_f1": fitted_model.macro_f1_score(y_test, y_pred),
        "weighted_precision": fitted_model.weighted_precision(y_test,
→ y_pred),
        "weighted_recall": fitted_model.weighted_recall(y_test, y_pred),

```

```

        "weighted_f1": fitted_model.weighted_f1_score(y_test, y_pred),
    },
)

# Below this is the old code provided in Regularization.ipynb
# which saves only the last fold model which is not correct
# I will refit the model with the best params and save the model later
signature = mlflow.models.infer_signature(X_train_original, pipeline.
↪predict(X_train_original))
mlflow.sklearn.log_model(pipeline, name="model", signature=signature)
mlflow.end_run()
# return models

```

```

[70]: # suppress overflow warnings for log
import warnings

with warnings.catch_warnings():
    warnings.simplefilter("ignore", RuntimeWarning)
    start_experiment(
        X_train,
        y_train_class,
        X_test,
        y_test_class,
        params_combinations=params_combinations,
        max_iter=5000,
    )

```

```

===== Experiment: 1 / 12 =====
Reg-Normal_TrainMethod-batch_LR-0.01
Loss at iteration 0 7.274561687315457
Loss at iteration 500 0.9725631997038706
Loss at iteration 1000 0.869082460435639
Loss at iteration 1500 0.8207499429042737
Loss at iteration 2000 0.7911486882121362
Loss at iteration 2500 0.7707920061958217
Loss at iteration 3000 0.7558022989507028
Loss at iteration 3500 0.7442427480914774
Loss at iteration 4000 0.7350268191120306
Loss at iteration 4500 0.7274933898843265
time taken: 2.2720985412597656
| Label | Precision | Recall | F1-Score |
|-----|-----|-----|-----|
| 0 | 0.8153 | 0.8252 | 0.8203 |
| 1 | 0.5674 | 0.5607 | 0.5640 |
| 2 | 0.5304 | 0.5633 | 0.5464 |
| 3 | 0.8034 | 0.7526 | 0.7772 |
| Macro Avg | 0.6791 | 0.6755 | 0.6770 |
| Weighted Avg | 0.6779 | 0.6745 | 0.6759 |

```

****Overall Accuracy:** 0.6745**

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See [`Handling Integers With Missing Values`](https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values) <<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values>>`_ for more details.

warnings.warn(

2025/10/05 11:07:45 WARNING mlflow.utils.environment: Failed to resolve installed pip version. ``pip`` will be added to conda.yaml environment spec without a version specifier.

View run Reg-Normal_TrainMethod-batch_LR-0.01 at: <https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231/runs/e244499ea8b34307ba1b7d086b2abbaf>

View experiment at:

<https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231>

===== Experiment: 2 / 12 =====

Reg-Normal_TrainMethod-batch_LR-0.001

Loss at iteration 0 5.183729645861908

Loss at iteration 500 1.5720900448696151

Loss at iteration 1000 1.3881123454277533

Loss at iteration 1500 1.2674127436914282

Loss at iteration 2000 1.1855719131892937

Loss at iteration 2500 1.1270535064301337

Loss at iteration 3000 1.083335583454253

Loss at iteration 3500 1.0495552158884878

Loss at iteration 4000 1.0224214897600636

Loss at iteration 4500 0.9999714450087692

time taken: 2.2775509357452393

| Label | Precision | Recall | F1-Score |

	0	1	2	3	Macro Avg	Weighted Avg
Accuracy	0.6989	0.7888	0.7412		0.5443	0.5495
Precision	0.3714	0.3341	0.3518		0.5517	0.5446
Recall	0.4458	0.4574	0.4515		0.5469	
F1 Score	0.6611	0.6263	0.6432			

****Overall Accuracy:** 0.5495**

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See <https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values> for more details.

warnings.warn(

2025/10/05 11:08:02 WARNING mlflow.utils.environment: Failed to resolve installed pip version. ``pip`` will be added to conda.yaml environment spec without a version specifier.

View run Reg-Normal_TrainMethod-batch_LR-0.001 at: <https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231/runs/c24994334ca445bc8d0ae7b24e537bb8>

View experiment at:

<https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231>

===== Experiment: 3 / 12 =====

Reg-Normal_TrainMethod-mini-batch_LR-0.01

Loss at iteration 0 7.118725228103791

Loss at iteration 500 0.9594986213682566

Loss at iteration 1000 0.869120439534371

Loss at iteration 1500 0.830124012180488

Loss at iteration 2000 0.7847347468530538

```

Loss at iteration 2500 0.7284817803637412
Loss at iteration 3000 0.7649257299134246
Loss at iteration 3500 0.759609523228107
Loss at iteration 4000 0.7331933374251739
Loss at iteration 4500 0.7215822390483262
time taken: 0.4195241928100586
| Label | Precision | Recall | F1-Score |
|-----|-----|-----|-----|
| 0 | 0.7963 | 0.8350 | 0.8152 |
| 1 | 0.5753 | 0.5444 | 0.5594 |
| 2 | 0.5293 | 0.6072 | 0.5656 |
| 3 | 0.8313 | 0.7132 | 0.7677 |
| Macro Avg | 0.6830 | 0.6749 | 0.6770 |
| Weighted Avg | 0.6814 | 0.6739 | 0.6757 |

```

****Overall Accuracy:** 0.6739**

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See ``Handling Integers With Missing Values`` <https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values> for more details.

warnings.warn(

2025/10/05 11:08:16 WARNING mlflow.utils.environment: Failed to resolve installed pip version. ``pip`` will be added to conda.yaml environment spec without a version specifier.

View run Reg-Normal_TrainMethod-mini-batch_LR-0.01 at: <https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231/runs/59c6d8712e95455f85b814e7381d170a>

View experiment at:

<https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231>

===== Experiment: 4 / 12 =====

Reg-Normal_TrainMethod-mini-batch_LR-0.001

Loss at iteration 0 3.6873782728636715

Loss at iteration 500 1.2093113996459475

Loss at iteration 1000 1.1371198754356509

Loss at iteration 1500 1.093183061289136

Loss at iteration 2000 1.1088701239135128

Loss at iteration 2500 1.042080709377463

Loss at iteration 3000 1.0166483426507145

Loss at iteration 3500 0.9834632056815512

Loss at iteration 4000 0.9668978970777572

Loss at iteration 4500 0.9617747992647016

time taken: 0.419755220413208

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

Label	Precision	Recall	F1-Score
-----	-----	-----	-----
0	0.7267	0.8131	0.7675
1	0.4268	0.4766	0.4503
2	0.3974	0.3101	0.3483
3	0.7077	0.6816	0.6944
Macro Avg	0.5646	0.5703	0.5651
Weighted Avg	0.5630	0.5713	0.5648

****Overall Accuracy:** 0.5713**

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See `Handling Integers With Missing Values

<<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing->

```

values>`_ for more details.
warnings.warn(
2025/10/05 11:08:29 WARNING mlflow.utils.environment: Failed to resolve
installed pip version. ``pip`` will be added to conda.yaml environment spec
without a version specifier.

View run Reg-Normal_TrainMethod-mini-batch_LR-0.001 at: https://mlflow.ml.brai
n.cs.ait.ac.th/#/experiments/655500363000293231/runs/64283aa17e6948169d97a7b3a50
94fca
View experiment at:
https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231
===== Experiment: 5 / 12 =====
Reg-Normal_TrainMethod-stochastic_LR-0.01
Loss at iteration 0 3.667017951539871
Loss at iteration 500 0.7235918410039879
Loss at iteration 1000 0.1133590822773061
Loss at iteration 1500 1.0130130962524586
Loss at iteration 2000 0.3356838401472585
Loss at iteration 2500 0.5445615647121955
Loss at iteration 3000 0.3743416241996252
Loss at iteration 3500 0.20491402956780463
Loss at iteration 4000 0.0007452194588127101
Loss at iteration 4500 0.0014538579247522032
time taken: 0.0660247802734375
| Label | Precision | Recall | F1-Score |
|-----|-----|-----|-----|
| 0 | 0.8852 | 0.3932 | 0.5445 |
| 1 | 0.3455 | 0.9276 | 0.5035 |
| 2 | 0.7333 | 0.0284 | 0.0547 |
| 3 | 0.8346 | 0.5711 | 0.6781 |
| Macro Avg | 0.6997 | 0.4801 | 0.4452 |
| Weighted Avg | 0.6929 | 0.4897 | 0.4472 |

**Overall Accuracy:** 0.4897

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not
fitted yet. Call 'fit' with appropriate arguments before using other methods
such as transform, predict, etc. This will raise an error in 1.8 instead of the
current warning.
warnings.warn(
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not
fitted yet. Call 'fit' with appropriate arguments before using other methods
such as transform, predict, etc. This will raise an error in 1.8 instead of the
current warning.
warnings.warn(
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains

```

integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See ``Handling Integers With Missing Values`` <https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values> for more details.

```
warnings.warn(
2025/10/05 11:08:44 WARNING mlflow.utils.environment: Failed to resolve
installed pip version. ``pip`` will be added to conda.yaml environment spec
without a version specifier.
```

View run Reg-Normal_TrainMethod-stochastic_LR-0.01 at: <https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231/runs/867555e8947b4098ae352fc04caf c4a4>

View experiment at:
<https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231>

===== Experiment: 6 / 12 =====

Reg-Normal_TrainMethod-stochastic_LR-0.001

```
Loss at iteration 0 0.8593659330338889
Loss at iteration 500 0.3266090556646508
Loss at iteration 1000 0.13994282277616793
Loss at iteration 1500 0.1570581031471575
Loss at iteration 2000 0.5217209817753939
Loss at iteration 2500 0.7112278431282966
Loss at iteration 3000 0.37429473740949465
Loss at iteration 3500 0.6593540736806475
Loss at iteration 4000 0.18846255003723306
Loss at iteration 4500 0.43102036077397354
time taken: 0.06551241874694824
```

Label	Precision	Recall	F1-Score
0	0.2810	0.4951	0.3585
1	0.2513	0.1145	0.1573
2	0.2921	0.1344	0.1841
3	0.4685	0.6263	0.5360
Macro Avg	0.3232	0.3426	0.3090
Weighted Avg	0.3201	0.3379	0.3049

****Overall Accuracy:** 0.3379**

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

```
warnings.warn(
```



```

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not
fitted yet. Call 'fit' with appropriate arguments before using other methods
such as transform, predict, etc. This will raise an error in 1.8 instead of the
current warning.
  warnings.warn(
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains
integer column(s). Integer columns in Python cannot represent missing values. If
your input data contains missing values at inference time, it will be encoded as
floats and will cause a schema enforcement error. The best way to avoid this
problem is to infer the model schema based on a realistic data sample (training
dataset) that includes missing values. Alternatively, you can declare integer
columns as doubles (float64) whenever these columns may have missing values. See
`Handling Integers With Missing Values
<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-
values>`_ for more details.
  warnings.warn(
2025/10/05 11:08:57 WARNING mlflow.utils.environment: Failed to resolve
installed pip version. ``pip`` will be added to conda.yaml environment spec
without a version specifier.

View run Reg-Normal_TrainMethod-stochastic_LR-0.001 at: https://mlflow.ml.brai
n.cs.ait.ac.th/#/experiments/655500363000293231/runs/af0a4f9cc1d04b10a58d3d05abd
9a9f1
View experiment at:
https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231
===== Experiment: 7 / 12 =====
Reg-Ridge_TrainMethod-batch_LR-0.01
Loss at iteration 0 3.8135539749933813
Loss at iteration 500 nan
Loss at iteration 1000 nan
Loss at iteration 1500 nan
Loss at iteration 2000 nan
Loss at iteration 2500 nan
Loss at iteration 3000 nan
Loss at iteration 3500 nan
Loss at iteration 4000 nan
Loss at iteration 4500 nan
time taken: 1.7147607803344727
| Label | Precision | Recall | F1-Score |
|-----|-----|-----|-----|
| 0 | 0.2564 | 1.0000 | 0.4081 |
| 1 | 0.0000 | 0.0000 | 0.0000 |
| 2 | 0.0000 | 0.0000 | 0.0000 |
| 3 | 0.0000 | 0.0000 | 0.0000 |
| Macro Avg | 0.0641 | 0.2500 | 0.1020 |
| Weighted Avg | 0.0657 | 0.2564 | 0.1046 |

```

```

**Overall Accuracy:** 0.2564

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not
fitted yet. Call 'fit' with appropriate arguments before using other methods
such as transform, predict, etc. This will raise an error in 1.8 instead of the
current warning.
    warnings.warn(
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not
fitted yet. Call 'fit' with appropriate arguments before using other methods
such as transform, predict, etc. This will raise an error in 1.8 instead of the
current warning.
    warnings.warn(
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains
integer column(s). Integer columns in Python cannot represent missing values. If
your input data contains missing values at inference time, it will be encoded as
floats and will cause a schema enforcement error. The best way to avoid this
problem is to infer the model schema based on a realistic data sample (training
dataset) that includes missing values. Alternatively, you can declare integer
columns as doubles (float64) whenever these columns may have missing values. See
`Handling Integers With Missing Values
<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-
values>`_ for more details.
    warnings.warn(
2025/10/05 11:09:12 WARNING mlflow.utils.environment: Failed to resolve
installed pip version. ``pip`` will be added to conda.yaml environment spec
without a version specifier.

View run Reg-Ridge_TrainMethod-batch_LR-0.01 at: https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231/runs/e93568fb59c140ea98f4b0b21121db41
View experiment at:
https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231
===== Experiment: 8 / 12 =====
Reg-Ridge_TrainMethod-batch_LR-0.001
Loss at iteration 0 7.626815150883831
Loss at iteration 500 nan
Loss at iteration 1000 nan
Loss at iteration 1500 nan
Loss at iteration 2000 nan
Loss at iteration 2500 nan
Loss at iteration 3000 nan
Loss at iteration 3500 nan
Loss at iteration 4000 nan
Loss at iteration 4500 nan
time taken: 1.7309787273406982
| Label | Precision | Recall | F1-Score |

```

	0	1	2	3	Macro Avg	Weighted Avg
Accuracy	0.2564	1.0000	0.4081	0.0000	0.0641	0.2564
Precision	0.0000	0.0000	0.0000	0.0000	0.0256	0.0657
Recall	0.0000	0.0000	0.0000	0.0000	0.1020	0.1046
F1 Score	0.0000	0.0000	0.0000	0.0000	0.0133	0.0133

****Overall Accuracy:** 0.2564**

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See <https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values> for more details.

warnings.warn(

2025/10/05 11:09:27 WARNING mlflow.utils.environment: Failed to resolve installed pip version. ``pip`` will be added to conda.yaml environment spec without a version specifier.

View run Reg-Ridge_TrainMethod-batch_LR-0.001 at: <https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231/runs/6c52ec9c6977461fa2b8f976c797dd6a>

View experiment at:

<https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231>

===== Experiment: 9 / 12 =====

Reg-Ridge_TrainMethod-mini-batch_LR-0.01

Loss at iteration 0 4.765793488395073

Loss at iteration 500 nan

Loss at iteration 1000 nan

Loss at iteration 1500 nan

Loss at iteration 2000 nan

```
Loss at iteration 2500 nan
Loss at iteration 3000 nan
Loss at iteration 3500 nan
Loss at iteration 4000 nan
Loss at iteration 4500 nan
time taken: 0.3908572196960449
```

```
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not
fitted yet. Call 'fit' with appropriate arguments before using other methods
such as transform, predict, etc. This will raise an error in 1.8 instead of the
current warning.
```

```
warnings.warn(
```

Label	Precision	Recall	F1-Score
0	0.2564	1.0000	0.4081
1	0.0000	0.0000	0.0000
2	0.0000	0.0000	0.0000
3	0.0000	0.0000	0.0000
Macro Avg	0.0641	0.2500	0.1020
Weighted Avg	0.0657	0.2564	0.1046

```
**Overall Accuracy:** 0.2564
```

```
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not
fitted yet. Call 'fit' with appropriate arguments before using other methods
such as transform, predict, etc. This will raise an error in 1.8 instead of the
current warning.
```

```
warnings.warn(
```

```
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains
integer column(s). Integer columns in Python cannot represent missing values. If
your input data contains missing values at inference time, it will be encoded as
floats and will cause a schema enforcement error. The best way to avoid this
problem is to infer the model schema based on a realistic data sample (training
dataset) that includes missing values. Alternatively, you can declare integer
columns as doubles (float64) whenever these columns may have missing values. See
`Handling Integers With Missing Values
<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-
values>`_ for more details.
```

```
warnings.warn(
```

```
2025/10/05 11:09:41 WARNING mlflow.utils.environment: Failed to resolve
installed pip version. ``pip`` will be added to conda.yaml environment spec
without a version specifier.
```

```
View run Reg-Ridge_TrainMethod-mini-batch_LR-0.01 at: https://mlflow.ml.brain.
cs.ait.ac.th/#/experiments/655500363000293231/runs/e3f29c05e01040af89163883ff348
7b2
```

View experiment at:
<https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231>

===== Experiment: 10 / 12 =====

Reg-Ridge_TrainMethod-mini-batch_LR-0.001

Loss at iteration 0 5.352148847666539

Loss at iteration 500 29.28164663891776

Loss at iteration 1000 12.871498933337742

Loss at iteration 1500 10.682161244490207

Loss at iteration 2000 21.050997843180294

Loss at iteration 2500 32.089853810767295

Loss at iteration 3000 6.857291516660692

Loss at iteration 3500 30.40665070883918

Loss at iteration 4000 21.319209377602395

Loss at iteration 4500 27.154398608529544

time taken: 0.4761180877685547

Label	Precision	Recall	F1-Score
-----	-----	-----	-----
0	0.6124	0.9320	0.7392
1	0.2500	0.0023	0.0046
2	0.4520	0.8398	0.5877
3	0.8833	0.5974	0.7127
Macro Avg	0.5494	0.5929	0.5111
Weighted Avg	0.5413	0.5831	0.5008

****Overall Accuracy:** 0.5831**

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

warnings.warn(

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See 'Handling Integers With Missing Values

<<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing->

```

values>`_ for more details.
warnings.warn(
2025/10/05 11:09:54 WARNING mlflow.utils.environment: Failed to resolve
installed pip version. ``pip`` will be added to conda.yaml environment spec
without a version specifier.

View run Reg-Ridge_TrainMethod-mini-batch_LR-0.001 at: https://mlflow.ml.brain
.cs.ait.ac.th/#/experiments/655500363000293231/runs/6c4aa581766b40b3bc43727919e6
a2d4
View experiment at:
https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231
===== Experiment: 11 / 12 =====
Reg-Ridge_TrainMethod-stochastic_LR-0.01
Loss at iteration 0 2.4216931036501195
Loss at iteration 500 0.6114278897586755
Loss at iteration 1000 0.8854149792887754
Loss at iteration 1500 2.097432451429847
Loss at iteration 2000 4.5151160515511215
Loss at iteration 2500 2.122624704527534
Loss at iteration 3000 1.1731055433033575
Loss at iteration 3500 2.499667909281991
Loss at iteration 4000 3.9046917225113913
Loss at iteration 4500 2.1020780472203895
time taken: 0.09350252151489258
| Label | Precision | Recall | F1-Score |
|-----|-----|-----|-----|
| 0 | 0.8076 | 0.5704 | 0.6686 |
| 1 | 0.4486 | 0.7757 | 0.5685 |
| 2 | 0.0000 | 0.0000 | 0.0000 |
| 3 | 0.5694 | 0.8632 | 0.6862 |
| Macro Avg | 0.4564 | 0.5523 | 0.4808 |
| Weighted Avg | 0.4612 | 0.5569 | 0.4851 |

**Overall Accuracy:** 0.5569

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not
fitted yet. Call 'fit' with appropriate arguments before using other methods
such as transform, predict, etc. This will raise an error in 1.8 instead of the
current warning.
warnings.warn(
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not
fitted yet. Call 'fit' with appropriate arguments before using other methods
such as transform, predict, etc. This will raise an error in 1.8 instead of the
current warning.
warnings.warn(
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains

```

integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See ``Handling Integers With Missing Values`` <https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values> for more details.

```
warnings.warn(
2025/10/05 11:10:08 WARNING mlflow.utils.environment: Failed to resolve
installed pip version. ``pip`` will be added to conda.yaml environment spec
without a version specifier.
```

View run Reg-Ridge_TrainMethod-stochastic_LR-0.01 at: <https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231/runs/993c25ed1bba47758b3f1ec94a381aa4>

View experiment at:
<https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231>

===== Experiment: 12 / 12 =====

Reg-Ridge_TrainMethod-stochastic_LR-0.001

Loss at iteration 0 5.31915522403085

Loss at iteration 500 4.876967835595418

Loss at iteration 1000 2.0292809729025594

Loss at iteration 1500 2.835883773427633

Loss at iteration 2000 1.65925666501297

Loss at iteration 2500 1.7550477073267188

Loss at iteration 3000 1.88040993328811

Loss at iteration 3500 2.215072184355758

Loss at iteration 4000 1.8261149711093494

Loss at iteration 4500 1.6861428444648399

time taken: 0.0899662971496582

Label	Precision	Recall	F1-Score
-----	-----	-----	-----
0	0.7864	0.6165	0.6912
1	0.3525	0.8037	0.4900
2	0.4000	0.0052	0.0102
3	0.7756	0.6184	0.6881
Macro Avg	0.5786	0.5110	0.4699
Weighted Avg	0.5752	0.5196	0.4729

****Overall Accuracy:**** 0.5196

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not fitted yet. Call 'fit' with appropriate arguments before using other methods such as transform, predict, etc. This will raise an error in 1.8 instead of the current warning.

```
warnings.warn(
```

```
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-  
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not  
fitted yet. Call 'fit' with appropriate arguments before using other methods  
such as transform, predict, etc. This will raise an error in 1.8 instead of the  
current warning.
```

```
warnings.warn(  
d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-  
packages\mlflow\types\utils.py:452: UserWarning: Hint: Inferred schema contains  
integer column(s). Integer columns in Python cannot represent missing values. If  
your input data contains missing values at inference time, it will be encoded as  
floats and will cause a schema enforcement error. The best way to avoid this  
problem is to infer the model schema based on a realistic data sample (training  
dataset) that includes missing values. Alternatively, you can declare integer  
columns as doubles (float64) whenever these columns may have missing values. See  
'Handling Integers With Missing Values  
<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-  
values>' for more details.
```

```
warnings.warn(  
2025/10/05 11:10:22 WARNING mlflow.utils.environment: Failed to resolve  
installed pip version. ``pip`` will be added to conda.yaml environment spec  
without a version specifier.
```

```
View run Reg-Ridge_TrainMethod-stochastic_LR-0.001 at: https://mlflow.ml.brain  
.cs.ait.ac.th/#/experiments/655500363000293231/runs/17b5fe8a625e4b2bb300d2995d48  
d056
```

```
View experiment at:  
https://mlflow.ml.brain.cs.ait.ac.th/#/experiments/655500363000293231
```

```
[ ]: mlflow.end_run()  # just in case there is some run stuck
```

```
[71]: runs = mlflow.search_runs(experiment_names=["st125923-a3"],  
    ↪search_all_experiments=False, output_format="pandas")
```

```
[72]: runs
```

```
[72]:
```

	run_id	experiment_id	status	\
0	17b5fe8a625e4b2bb300d2995d48d056	655500363000293231	FINISHED	
1	993c25ed1bba47758b3f1ec94a381aa4	655500363000293231	FINISHED	
2	6c4aa581766b40b3bc43727919e6a2d4	655500363000293231	FINISHED	
3	e3f29c05e01040af89163883ff3487b2	655500363000293231	FINISHED	
4	6c52ec9c6977461fa2b8f976c797dd6a	655500363000293231	FINISHED	
5	e93568fb59c140ea98f4b0b21121db41	655500363000293231	FINISHED	
6	af0a4f9cc1d04b10a58d3d05abd9a9f1	655500363000293231	FINISHED	
7	867555e8947b4098ae352fc04cafc4a4	655500363000293231	FINISHED	
8	64283aa17e6948169d97a7b3a5094fca	655500363000293231	FINISHED	
9	59c6d8712e95455f85b814e7381d170a	655500363000293231	FINISHED	
10	c24994334ca445bc8d0ae7b24e537bb8	655500363000293231	FINISHED	
11	e244499ea8b34307ba1b7d086b2abbaf	655500363000293231	FINISHED	

	artifact_uri \
0	mlflow-artifacts:/655500363000293231/17b5fe8a6...
1	mlflow-artifacts:/655500363000293231/993c25ed1...
2	mlflow-artifacts:/655500363000293231/6c4aa5817...
3	mlflow-artifacts:/655500363000293231/e3f29c05e...
4	mlflow-artifacts:/655500363000293231/6c52ec9c6...
5	mlflow-artifacts:/655500363000293231/e93568fb5...
6	mlflow-artifacts:/655500363000293231/af0a4f9cc...
7	mlflow-artifacts:/655500363000293231/867555e89...
8	mlflow-artifacts:/655500363000293231/64283aa17...
9	mlflow-artifacts:/655500363000293231/59c6d8712...
10	mlflow-artifacts:/655500363000293231/c24994334...
11	mlflow-artifacts:/655500363000293231/e244499ea...

	start_time	end_time \
0	2025-10-05 04:10:10.406000+00:00	2025-10-05 04:10:23.467000+00:00
1	2025-10-05 04:09:56.848000+00:00	2025-10-05 04:10:10.019000+00:00
2	2025-10-05 04:09:42.929000+00:00	2025-10-05 04:09:56.452000+00:00
3	2025-10-05 04:09:29.397000+00:00	2025-10-05 04:09:42.547000+00:00
4	2025-10-05 04:09:14.481000+00:00	2025-10-05 04:09:29.014000+00:00
5	2025-10-05 04:08:59.606000+00:00	2025-10-05 04:09:14.103000+00:00
6	2025-10-05 04:08:46.418000+00:00	2025-10-05 04:08:59.226000+00:00
7	2025-10-05 04:08:31.727000+00:00	2025-10-05 04:08:46.040000+00:00
8	2025-10-05 04:08:18.028000+00:00	2025-10-05 04:08:31.350000+00:00
9	2025-10-05 04:08:04.194000+00:00	2025-10-05 04:08:17.634000+00:00
10	2025-10-05 04:07:47.441000+00:00	2025-10-05 04:08:03.812000+00:00
11	2025-10-05 04:07:27.639000+00:00	2025-10-05 04:07:47.058000+00:00

	metrics.class_1_f1	metrics.macro_f1	metrics.class_3_recall \
0	0.490028	0.469882	0.618421
1	0.568493	0.480812	0.863158
2	0.004630	0.511055	0.597368
3	0.000000	0.102031	0.000000
4	0.000000	0.102031	0.000000
5	0.000000	0.102031	0.000000
6	0.157303	0.308983	0.626316
7	0.503488	0.445219	0.571053
8	0.450331	0.565125	0.681579
9	0.559424	0.676970	0.713158
10	0.351784	0.546930	0.626316
11	0.564042	0.676962	0.752632

	metrics.class_1_precision	...	metrics.class_0_f1 \
0	0.352459	...	0.691156
1	0.448649	...	0.668563
2	0.250000	...	0.739172

3	0.000000	...	0.408123
4	0.000000	...	0.408123
5	0.000000	...	0.408123
6	0.251282	...	0.358524
7	0.345518	...	0.544538
8	0.426778	...	0.767468
9	0.575309	...	0.815166
10	0.371429	...	0.741163
11	0.567376	...	0.820265

	metrics.class_2_recall	metrics.class_0_precision	params.learning_rate \
0	0.005168	0.786378	0.001
1	0.000000	0.807560	0.01
2	0.839793	0.612440	0.001
3	0.000000	0.256378	0.01
4	0.000000	0.256378	0.001
5	0.000000	0.256378	0.01
6	0.134367	0.280992	0.001
7	0.028424	0.885246	0.01
8	0.310078	0.726681	0.001
9	0.607235	0.796296	0.01
10	0.457364	0.698925	0.001
11	0.563307	0.815348	0.01

	params.model_type	params.train_method	tags.mlflow.user \
0	Ridge	stochastic	admin
1	Ridge	stochastic	admin
2	Ridge	mini-batch	admin
3	Ridge	mini-batch	admin
4	Ridge	batch	admin
5	Ridge	batch	admin
6	Normal	stochastic	admin
7	Normal	stochastic	admin
8	Normal	mini-batch	admin
9	Normal	mini-batch	admin
10	Normal	batch	admin
11	Normal	batch	admin

	tags.mlflow.source.name \
0	d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...
1	d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...
2	d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...
3	d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...
4	d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...
5	d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...
6	d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...
7	d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...

```

8 d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...
9 d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...
10 d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...
11 d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\s...

```

	tags.mlflow.runName	tags.mlflow.source.type
0	Reg-Ridge_TrainMethod-stochastic_LR-0.001	LOCAL
1	Reg-Ridge_TrainMethod-stochastic_LR-0.01	LOCAL
2	Reg-Ridge_TrainMethod-mini-batch_LR-0.001	LOCAL
3	Reg-Ridge_TrainMethod-mini-batch_LR-0.01	LOCAL
4	Reg-Ridge_TrainMethod-batch_LR-0.001	LOCAL
5	Reg-Ridge_TrainMethod-batch_LR-0.01	LOCAL
6	Reg-Normal_TrainMethod-stochastic_LR-0.001	LOCAL
7	Reg-Normal_TrainMethod-stochastic_LR-0.01	LOCAL
8	Reg-Normal_TrainMethod-mini-batch_LR-0.001	LOCAL
9	Reg-Normal_TrainMethod-mini-batch_LR-0.01	LOCAL
10	Reg-Normal_TrainMethod-batch_LR-0.001	LOCAL
11	Reg-Normal_TrainMethod-batch_LR-0.01	LOCAL

[12 rows x 32 columns]

```

[73]: runs[["tags.mlflow.runName", "metrics.macro_f1", "metrics.weighted_f1",
↪ "metrics.accuracy"]].sort_values(
      by="metrics.macro_f1",
      ascending=False,
      )

```

	tags.mlflow.runName	metrics.macro_f1	\
9	Reg-Normal_TrainMethod-mini-batch_LR-0.01	0.676970	
11	Reg-Normal_TrainMethod-batch_LR-0.01	0.676962	
8	Reg-Normal_TrainMethod-mini-batch_LR-0.001	0.565125	
10	Reg-Normal_TrainMethod-batch_LR-0.001	0.546930	
2	Reg-Ridge_TrainMethod-mini-batch_LR-0.001	0.511055	
1	Reg-Ridge_TrainMethod-stochastic_LR-0.01	0.480812	
0	Reg-Ridge_TrainMethod-stochastic_LR-0.001	0.469882	
7	Reg-Normal_TrainMethod-stochastic_LR-0.01	0.445219	
6	Reg-Normal_TrainMethod-stochastic_LR-0.001	0.308983	
3	Reg-Ridge_TrainMethod-mini-batch_LR-0.01	0.102031	
4	Reg-Ridge_TrainMethod-batch_LR-0.001	0.102031	
5	Reg-Ridge_TrainMethod-batch_LR-0.01	0.102031	

	metrics.weighted_f1	metrics.accuracy
9	0.675725	0.673927
11	0.675874	0.674549
8	0.564781	0.571251
10	0.544553	0.549471
2	0.500805	0.583074

1	0.485075	0.556938
0	0.472888	0.519602
7	0.447236	0.489732
6	0.304895	0.337897
3	0.104634	0.256378
4	0.104634	0.256378
5	0.104634	0.256378

```
[74]: best_run_id = runs.sort_values("metrics.macro_f1", ascending=False).
      ↪iloc[0]["run_id"]
      best_run_name = runs.sort_values("metrics.macro_f1", ascending=False).
      ↪iloc[0]["tags.mlflow.runName"]
```

```
[75]: print(best_run_id)
      print(best_run_name)
```

```
59c6d8712e95455f85b814e7381d170a
Reg-Normal_TrainMethod-mini-batch_LR-0.01
```

2.0.1 After Registering the model in MLflow UI

```
[77]: client = MlflowClient()
```

```
[78]: client.set_registered_model_alias("st125923-a3-model", "staging", 1)
```

```
[ ]: # Try loading the model from model registry
      loaded_model = mlflow.sklearn.load_model("models:/st125923-a3-model@staging")
```

```
[80]: loaded_model
```

```
[80]: Pipeline(steps=[('preprocess',
                        ColumnTransformer(remainder='passthrough',
                                          transformers=[('brand_le',
                                                         Pipeline(steps=[('pre_imputer',
                                                         SimpleImputer(strategy='most_frequent')),
                                                         ('encoder',
                                                         OrdinalEncoder(handle_unknown='use_encoded_value',
                                                         unknown_value=nan)),
                                                         ('post_imputer',
                                                         SimpleImputer(strategy='most_frequent'))])),
                                                         ['brand']],
                                                         ('fuel_le',
                                                         Pipeline(steps=[('...
                                                         ('engine_imputer',
                                                         SimpleImputer(strategy='median'),
                                                         ['engine']],
                                                         ('max_power_imputer',
                                                         SimpleImputer(strategy='median'),
```

```

        ['max_power']],
        ('seats_imputer',
         SimpleImputer(strategy='most_frequent'),
         ['seats'])))),
        ('scaler',
         StandardScaler()))],
        ['year', 'km_driven',
         'mileage', 'engine',
         'max_power', 'seats'])])),
        ('model',
         <__main__.LogisticRegression object at 0x000001ED4AA0D9D0>))]

```

```

[81]: # Recheck if the loaded model works correctly
print(classification_report(y_test_class, loaded_model.predict(X_test)))

```

	precision	recall	f1-score	support
0	0.80	0.83	0.82	412
1	0.58	0.54	0.56	428
2	0.53	0.61	0.57	387
3	0.83	0.71	0.77	380
accuracy			0.67	1607
macro avg	0.68	0.67	0.68	1607
weighted avg	0.68	0.67	0.68	1607

```

d:\AT82.03-ML-Car-Price-Prediction\.venv\Lib\site-
packages\sklearn\pipeline.py:61: FutureWarning: This Pipeline instance is not
fitted yet. Call 'fit' with appropriate arguments before using other methods
such as transform, predict, etc. This will raise an error in 1.8 instead of the
current warning.
  warnings.warn(

```

```

[ ]: client.transition_model_version_stage

```

3 A3 Summary

3.1 About the model performance

- Multinomial Logistic Regression with 4 classes performed not that good with the best macro F1-score of around 0.68
- Normal model without regularization performed better than Ridge regularization overall
- Higher learning rate (0.01) somehow performed better than lower learning rate (0.001), which is counter-intuitive especially compared to last assignment on linear regression where 0.01 always diverged
- For Ridge regularization models, they performed not that good overall, It is possible that I choose too many features (all features) and the regularization term penalizes too much as

Ridge tends to hammer down the big theta which might occur in some datasets with a few important features

3.2 About the MLflow

- MLflow is a great tool to track experiments and manage models but the version mismatch between local and server is confusing and the functionality of staging/production is not that clear because it's already deprecated in the newer version and the documentation is not that helpful.
- I found workaround to stage the model by using alias name instead of setting the stage of the model which is deprecated. If I try to set the stage of the model with `client.transition_model_version_stage` it will run for 4 minutes and show error `MlflowException: API request to http://mlflow.ml.brain.cs.ait.ac.th/api/2.0/mlflow/model-versions/transition-stage failed with exception HTTPSConnectionPool(host='mlflow.ml.brain.cs.ait.ac.th', port=443): Max retries exceeded with url: /api/2.0/mlflow/model-versions/transition-stage (Caused by ResponseError('too many 500 error responses'))`. So I just use alias name to represent the stage instead of using the stage feature itself by using `client.set_registered_model_alias("st125923-a3-model", "staging", 1)` and `loaded_model = mlflow.sklearn.load_model("models:/st125923-a3-model@staging")`.
- the reason why I use `mlflow.sklearn` to load model is because I add the model into sklearn pipeline, so I can input raw data into the model without preprocessing it first as the pipeline will handle the preprocessing step which is very convenient for deployment

3.3 About GitHub Actions

- I have set up GitHub Actions to automatically run the test, then build and push new docker images (both frontend and backend because I don't use Dash) to DockerHub, then SSH and copy my `docker-compose-deploy.yml` file to the server and create `.env` file on the server and run it to pull the newly pushed docker images from DockerHub and restart the existing containers.